

Comenzado el	lunes, 13 de enero de 2025, 16:50
Estado	Finalizado
Finalizado en	lunes, 13 de enero de 2025, 17:23
Tiempo empleado	32 minutos 59 segundos
Puntos	17,00/25,00
Calificación	6,80 de 10,00 (68%)

Pregunta 1

Correcta

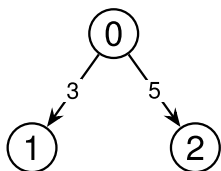
Se puntúa 1,00 sobre 1,00

Al aplicar el algoritmo de Dijkstra sobre un digrafo valorado $G = (V, A)$ de costes no negativos con al menos una arista, ¿cuáles de las siguientes expresiones acotan durante todo el algoritmo el tamaño de la cola de prioridad de vértices pendientes?

Seleccione una o más de una:

- ☐ a. $|A| - 1$
- ☒ b. $|V|$ ✓
- ☒ c. $|A|$ ✓
- ☒ d. $|V| - 1$ ✓

El algoritmo de Dijkstra almacena en la cola de prioridad los vértices del grafo que ya han sido alcanzados pero cuya distancia aún no es definitiva. Inicialmente, el único elemento de la cola es el vértice inicial, que ya no estará en las siguientes iteraciones. En adelante por tanto habrá como mucho $|V| - 1$ elementos en la cola. Por otro lado, para que un elemento se haya insertado en la cola ha tenido que visitarse una arista que lo uniese con un vértice ya visitado, por lo que no puede haber más de $|A|$ elementos en la cola. No es cierto que $|A| - 1$ acote el tamaño de la cola como muestra el siguiente grafo donde 0 es el vértice origen.



Las respuestas correctas son: $|V|$, $|A|$, $|V| - 1$

Pregunta 2

Correcta

Se puntúa 1,00 sobre 1,00

¿Qué complejidad tendría un algoritmo para calcular el grado de cada uno de los vértices de un grafo no dirigido representado mediante *matriz de adyacencia* si el grafo tiene V vértices y A aristas?

Seleccione una:

- ☐ a. $O(V)$
- ☐ b. $O(V * A)$
- ☐ c. $O(V + A)$
- ☒ d. $O(V^2)$ ✓

Calcular el grado de un vértice tiene coste en $O(V)$ (hay que recorrer completamente una fila o una columna de la matriz). Y esta operación se repite para cada vértice. El coste total está en $O(V^2)$.

La respuesta correcta es: $O(V^2)$

Pregunta 3

Correcta

Se puntúa 1,00 sobre 1,00

¿Qué complejidad tendría un algoritmo para calcular el *grado de entrada* de cada uno de los vértices de un grafo dirigido representado mediante *listas de adyacentes* si el grafo tiene V vértices y A aristas?

Seleccione una:

- ☐ a. $O(V * A)$
- ☐ b. $O(V^2)$
- ☐ c. $O(V)$
- ☒ d. $O(V + A)$ ✓

La forma más eficiente es calcular a la vez los grados de entrada de todos los vértices recorriendo todas las listas de adyacentes, y por cada nuevo adyacente encontrado incrementamos en 1 su grado de entrada. Hay V listas y el número total de nodos en todas ellas es A , por lo que el coste total está en $O(V + A)$.

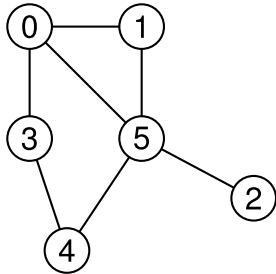
La respuesta correcta es: $O(V + A)$

Pregunta 4

Incorrecta

Se puntúa 0,00 sobre 1,00

¿En qué orden se visitarían los vértices de este grafo si realizamos un recorrido en profundidad desde el vértice 4? Escribe los identificadores de los vértices separados por espacios en el orden en que son visitados. Supón que los vértices en las listas de adyacentes están ordenados de menor a mayor.



Respuesta: 4 3 5 0 1 2



Los vértices se recorren en este orden: 4 3 0 1 5 2.

La respuesta correcta es: 4 3 0 1 5 2

Pregunta 5

Correcta

Se puntúa 1,00 sobre 1,00

El algoritmo voraz para resolver el problema de la mochila con n objetos que se pueden fraccionar y peso límite también n tiene un coste en:

Seleccione una:

- ☐ a. $O(n^2)$
- ☐ b. $O(n)$
- ☐ c. $O(n^2 \log n)$
- ☒ d. $O(n \log n)$ ✓

El coste del algoritmo voraz que resuelve el problema de la mochila real es independiente del peso límite. El coste está dominado por la fase de ordenar los objetos de mayor a menor valor por unidad de peso, que está en $O(n \log n)$ siendo n el número de objetos. La segunda fase, de selección de los objetos, los recorre todos en el caso peor y tiene coste en $O(n)$.

La respuesta correcta es: $O(n \log n)$

Pregunta 6

Correcta

Se puntúa 1,00 sobre 1,00

En el algoritmo de programación dinámica que resuelve el problema del producto encadenado de matrices, podemos rellenar la matriz:

Seleccione una:

- ☐ a. por diagonales comenzando en la diagonal del elemento del extremo superior derecho.
- ☐ b. de arriba abajo y de izquierda a derecha.
- ☐ c. de arriba abajo y de derecha a izquierda.
- ☒ d. por diagonales comenzando en la diagonal encima de la principal. ✓

Puesto que que cada posición (i, j) depende de las posiciones (i, k) y $(k + 1, j)$ tales que $i \leq k \leq j - 1$, las cuales están respectivamente en su misma fila a la izquierda hasta la diagonal principal, y en la misma columna pero en filas inferiores hasta la diagonal principal, una posible forma de rellenar la matriz es por diagonales comenzando en la diagonal encima de la principal. El resto de opciones no garantiza que estén rellenas las posiciones necesarias. Sí podría rellenarse, sin embargo, de abajo arriba y de izquierda a derecha.

La respuesta correcta es: por diagonales comenzando en la diagonal encima de la principal.

Pregunta 7

Sin contestar

Se puntúa como 0 sobre 1,00

El coste del algoritmo que ordena N valores insertándolos primero en un árbol binario de búsqueda y recorriéndolo completamente a continuación para producir la lista ordenada de dichos elementos está en el caso peor en

Seleccione una:

- ☐ a. $O(N^2)$
- ☐ b. $O(\log N)$
- ☐ c. $O(N)$
- ☐ d. $O(N \log N)$

Si el árbol no se autoequilibra (no es AVL, por ejemplo), en el caso peor cada inserción puede tener un coste lineal respecto a N . Por tanto la construcción del árbol tiene un coste en $O(N^2)$. Producir la lista ordenada requiere un recorrido en inorden, con un coste en $O(N)$.

La respuesta correcta es: $O(N^2)$

Pregunta 8

Sin contestar

Se puntúa como 0 sobre 1,00

¿Cuál de las siguientes afirmaciones sobre el coste en tiempo de la estrategia voraz para resolver un problema en el que queremos maximizar el beneficio de n tareas de igual duración con plazo fijo, es correcta?

Seleccione una:

- ☐ a. $O(n * n)$
- ☐ b. $O(n \log n)$
- ☐ c. $O(n)$
- ☐ d. Ninguna de las anteriores.

- a. Falso. Incorrecta. El mayor coste en tiempo viene de ordenar las tareas de menor a mayor por su beneficio, pero una ordenación eficiente es del orden de $O(n \log n)$. El resto de operaciones (incluyendo unir y buscar en conjuntos disjuntos) son, a efectos prácticos, constantes de ahí que no añadan más coste en tiempo.
- b. Cierto. No podemos asumir que las tareas vienen ordenadas por su beneficio, la mayor parte del coste en tiempo viene de ordenarlas de menor a mayor lo que supone un coste $O(n \log n)$, el resto de operaciones (incluyendo unir y buscar en conjuntos disjuntos) son, a efectos prácticos, constantes de ahí que no añadan más coste en tiempo.
- c. Falso. Incorrecta. No podemos asumir que las tareas están ordenadas de menor a mayor, por lo tanto el coste de ordenarlas será $O(n \log n)$. El resto de operaciones (incluyendo unir y buscar en conjuntos disjuntos) son, a efectos prácticos, constantes de ahí que no añadan más coste en tiempo.
- d. Falso. La respuesta correcta es: $O(n \log n)$

La respuesta correcta es: $O(n \log n)$ **Pregunta 9**

Correcta

Se puntúa 1,00 sobre 1,00

El algoritmo de programación dinámica para resolver el problema de la mochila con n objetos que no se pueden partir y peso máximo de la mochila n tiene un coste en $O(n^2)$.

Seleccione una:

- ☒ a. Verdadero ✓
- ☐ b. Falso

Verdadero. El coste en tiempo del algoritmo de programación dinámica que resuelve el problema de la mochila está en $O(m * P)$ siendo m el número de objetos y P el peso límite de la mochila. Por tanto, si el número de objetos y el peso límite son iguales, n según el enunciado de la pregunta, el coste estará en $O(n^2)$.

La respuesta correcta es: Verdadero

Pregunta 10

Correcta

Se puntúa 1,00 sobre 1,00

La función

```
void funcion(vector<int> & v) {  
    for (int i = v.size() - 1; i > 0; --i) {  
        flotar_max(v, v.size(), i);  
    }  
}
```

Seleccione una:

- ☐ a. siempre deja el vector sin modificar.
- ☐ b. convierte un montículo de máximos en uno de mínimos.
- ☐ c. convierte un vector cualquiera en un montículo.
- ☒ d. mueve los elementos del vector, pero no lo convierte necesariamente en un montículo. ✓

Un vector puede convertirse en un montículo de dos maneras distintas:

- recorriendo los elementos de izquierda a derecha y *flotando* cada elemento entre los anteriores (ya procesados), o
- recorriendo la primera mitad de los elementos de derecha a izquierda, *hundiendo* cada elemento entre los siguientes, que ya forman montículos.

Aquí se han mezclado las dos ideas, lo que no garantiza que el resultado final sea un montículo, aunque haya elementos que cambien de lugar.

La respuesta correcta es: mueve los elementos del vector, pero no lo convierte necesariamente en un montículo.

Pregunta 11

Correcta

Se puntúa 1,00 sobre 1,00

El problema de la multiplicación encadenada de n matrices de forma óptima puede resolverse con una complejidad en tiempo en:

Seleccione una:

- ☐ a. $O(n^2)$
- ☒ b. $O(n^3)$ ✓
- ☐ c. $O(m^2)$ siendo m el número de filas y columnas de las n matrices
- ☐ d. $O(m^3)$ siendo m el número de filas y columnas de las n matrices

El algoritmo calcula para cada pareja de índices i, j tales que $i \leq j$ el número mínimo de multiplicaciones básicas para multiplicar las matrices de la i a la j . El cálculo de cada valor es lineal en n porque requiere considerar las distintas formas de colocar los paréntesis externos para realizar la última multiplicación. Puesto que la cantidad de parejas a calcular es del $O(n^2)$, el coste está en $O(n^3)$.

La respuesta correcta es: $O(n^3)$

Pregunta 12

Sin contestar

Se puntúa como 0 sobre 1,00

Al finalizar su ejecución un algoritmo de ramificación y poda la cola de prioridad:

Seleccione una:

- ☐ a. solo puede contener nodos no prometedores.
- ☐ b. siempre queda vacía.
- ☐ c. puede contener nodos no factibles siempre que sean prometedores.
- ☐ d. puede contener nodos prometedores y no prometedores.

El algoritmo de ramificación y poda puede detenerse o bien porque la cola de prioridad queda vacía o bien porque la prioridad del más prioritario de la cola es peor (menor o mayor dependiendo de si es un problema de maximización o minimización) que el valor de la mejor solución encontrada hasta el momento. Por tanto, no siempre queda vacía y en caso de que no lo esté, todos los nodos que permanecen en ella son no prometedores. Por otra parte, en la cola de prioridad nunca entran nodos no factibles. Así que la respuesta correcta es que solo puede contener nodos no prometedores.

La respuesta correcta es: solo puede contener nodos no prometedores.

Pregunta 13

Correcta

Se puntúa 1,00 sobre 1,00

Supongamos una función recursiva que está definida de la siguiente manera:

$$f(1) = e_1$$

$$f(i) = \max(e_i, f(i-1) + e_i) \text{ para } i > 1$$

siendo $e_1, \dots, e_n$ expresiones numéricas y que queremos implementar un algoritmo de programación dinámica ascendente que calcule $f(n)$. ¿Qué cantidad de memoria adicional óptima necesita el algoritmo?

Seleccione una:

- ☐ a. $\theta(n^2)$
- ☐ b. $\theta(n \log n)$
- ☒ c. $\theta(1)$ ✓
- ☐ d. $\theta(n)$

Basta con utilizar una variable para guardar el valor de $f(i)$ que vamos calculando (desde 1 hasta n por ser ascendente), luego la respuesta correcta es $\theta(1)$.

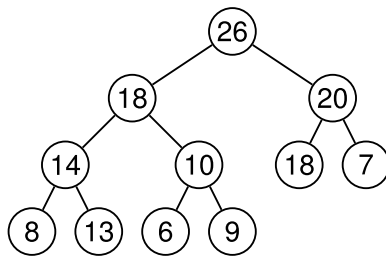
La respuesta correcta es: $\theta(1)$

Pregunta 14

Correcta

Se puntúa 1,00 sobre 1,00

En este montículo de máximos, ¿qué valores pueden haber sido el último en insertarse?



Seleccione una:

- ☐ a. 18, 10 y 9.
- ☐ b. 26, 18, 10 y 9.
- ☒ c. 10 y 9. ✓
- ☐ d. Solamente el 9.

El último elemento insertado se añadió en la hoja más a la derecha del último nivel y después fue flotado si hacía falta hacia la raíz. Puede haber sido el 9 y no haber necesitado ser flotado. Puede haber sido el 10 y haberse intercambiado con el 9, que ocupaba su posición. Pero no puede ser el 18, porque entonces el 10 ocuparía su posición antes de la última inserción, y el 10 no puede ser padre del 14 en un montículo de máximos. Por el mismo motivo no puede ser el 26 el último elemento insertado.

La respuesta correcta es: 10 y 9.

Pregunta 15

Sin contestar

Se puntúa como 0 sobre 1,00

Sea el problema de la mochila entera con los objetos siguientes, donde la primera componente es el valor y la segunda el peso: $(10, 1)$, $(12, 4)$, $(18, 9)$, $(4, 4)$. Si la capacidad de la mochila es 12, indica qué afirmaciones son correctas respecto al algoritmo de ramificación y poda que utiliza el algoritmo voraz para calcular la prioridad de los nodos:

Seleccione una o más de una:

- ☐ a. El número de posibles soluciones para este ejemplo es 16 y el número de nodos que pueden generarse es 31.
- ☐ b. El número de nodos generados durante la ejecución del algoritmo incluyendo la raíz es 12.
- ☐ c. El número de nodos generados durante la ejecución del algoritmo incluyendo la raíz es 9.
- ☐ d. El número de posibles soluciones para este ejemplo es 24 y el número de nodos que pueden generarse es 33.

El espacio de soluciones para este problema es un árbol binario completo de altura 5 (el nodo raíz y un nivel para cada uno de los 4 objetos) así que el número de nodos que podrían generarse son $2^5 - 1 = 31$ y el número de posibles soluciones, que es el número de nodos en el último nivel es $2^4 = 16$.

Representando en cada nodo el valor, peso y estimación de la solución parcial por una tripla $(valor, peso, estimación)$, veamos los nodos que se generan:

1-Inicialmente en la cola solo está el nodo raíz $nodo_0 = (0, 0, 36)$.

2- Sale ese nodo y entran $nodo_1 = (10, 1, 36)$ y $nodo_2 = (0, 0, 28)$.

3- Sale $nodo_1$ y entran $nodo_3 = (22, 5, 36)$ y $nodo_4 = (10, 1, 30)$.

4- Sale $nodo_3$ y entra $nodo_5 = (22, 5, 26)$.

5- Sale $nodo_4$ y entran $nodo_6 = (28, 10, 30)$ y $nodo_7 = (10, 1, 14)$.

6- Sale $nodo_6$ y su hijo $nodo_8 = (28, 10, 28)$ es la primera solución.

7- Puesto que el valor de esta solución es mayor o igual que las prioridades de los nodos que están en la cola, el algoritmo termina.

Por tanto, se generan en total 9 nodos incluyendo a la raíz.

Las respuestas correctas son: El número de posibles soluciones para este ejemplo es 16 y el número de nodos que pueden generarse es 31., El número de nodos generados durante la ejecución del algoritmo incluyendo la raíz es 9.

Pregunta 16

Correcta

Se puntúa 1,00 sobre 1,00

¿Cuáles de los siguientes podrían ser el resultado de realizar *uniones rápidas por tamaño* en una partición de 10 elementos?

Seleccione una o más de una:

- ☐ a.
- | | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| p | 5 | 5 | 5 | 9 | 5 | 9 | 8 | 2 | 9 | 9 |
- ☐ b.
- | | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| p | 4 | 4 | 1 | 0 | 8 | 0 | 0 | 4 | 6 | 4 |
- ☒ c.
- | | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| p | 0 | 1 | 2 | 1 | 1 | 8 | 6 | 7 | 8 | 9 |
- ☐ d.
- | | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| p | 9 | 9 | 3 | 0 | 0 | 2 | 8 | 6 | 8 | 9 |

✓ Verdadero. Si inicialmente hay 10 clases unitarias, bastaría con hacer por ejemplo las uniones 4-1, 3-1 y 5-8.

- a. Falso. El subárbol de raíz 5 tiene 5 elementos que han pasado a formar parte de esa subclase antes de pasar a unirse con su padre, el 9, ya que en caso contrario no apuntarían al 5. Sin embargo, no es posible que dicho subárbol se haya colocado como hijo de 9: puesto que el resto del árbol solo tiene 4 elementos la unión por tamaños habría colocado al 9 como hijo de 5 y no al revés.
- b. Falso. Un elemento de cada clase ha de ser el representante de la misma. Dicho elemento contiene en el vector su propio valor. En este ejemplo no hay ningún elemento que cumpla dicha propiedad.
- c. Verdadero. Si inicialmente hay 10 clases unitarias, bastaría con hacer por ejemplo las uniones 4-1, 3-1 y 5-8.
- d. Falso. Utilizando uniones rápidas por tamaño un subárbol de tres elementos siempre tendrá altura 2 y no 3. En este caso hay un subárbol de tres elementos con altura 3, el formado por los elementos 8, 6 y 7.

La respuesta correcta es: p

	0	1	2	3	4	5	6	7	8	9
p	0	1	2	1	1	8	6	7	8	9

Pregunta 17

Correcta

Se puntúa 1,00 sobre 1,00

¿Qué complejidad tendría un algoritmo para calcular el grado de cada uno de los vértices de un grafo no dirigido representado mediante *listas de adyacentes* si el grafo tiene V vértices y A aristas?

Seleccione una:

- ☒ a. $O(V)$ ✓
- ☐ b. $O(V \cdot A)$
- ☐ c. $O(V^2)$
- ☐ d. $O(V + A)$

Calcular el grado de un vértice tiene coste en $O(1)$ (los adyacentes están ya almacenados en una lista, y solamente necesitamos la longitud de la lista, no recorrerla). Y esta operación se repite para cada vértice. El coste total está en $O(V)$.

La respuesta correcta es: $O(V)$

Pregunta 18

Incorrecta

Se puntúa 0,00 sobre 1,00

¿Cuál es el coste de insertar en una cola de prioridad de mínimos inicialmente vacía implementada mediante un montículo de mínimos una secuencia de N elementos ordenada de menor a mayor?

Seleccione una:

- ☐ a. $O(1)$
- ☐ b. $O(N)$
- ☒ c. $O(N \log N)$ ✖
- ☐ d. $O(\log N)$

Cada nuevo elemento que se inserta es mayor que los que ya hay en el montículo, por lo que al colocarlo en la última posición no es necesario flotarlo, teniendo cada inserción un coste constante.

La respuesta correcta es: $O(N)$

Pregunta 19

Correcta

Se puntúa 1,00 sobre 1,00

Supongamos que el algoritmo de Floyd va a empezar a rellenar la matriz k -ésima, es decir, la que corresponde a considerar los vértices del 1 al k como vértices intermedios. Indica qué afirmaciones son correctas en un grafo sin ciclos de coste negativo:

Seleccione una o más de una:

- ☒ a. La diagonal principal no cambia. ✔
- ☒ b. Para rellenar cada posición se necesitan a lo sumo tres posiciones de la matriz $(k-1)$ -ésima. ✔
- ☒ c. La columna k no cambia. ✔
- ☒ d. La fila k no cambia. ✔

Se puede demostrar que en la iteración k -ésima los valores de la fila k y de la columna k no se modifican, ya que poder pasar por el vértice k para construir caminos desde o hasta k no permite reducir el coste de los caminos mínimos obtenidos con los vértices 1 a $k-1$ como intermedios si no hay ciclos de coste negativo.

Los valores de las diagonales valen 0 en todas las iteraciones si no hay ciclos de coste negativo.

En la iteración k -ésima cada posición (i, j) de la matriz necesita los valores (i, k) , (k, j) y el propio (i, j) de la matriz $(k-1)$ -ésima.

Las respuestas correctas son: La diagonal principal no cambia., Para rellenar cada posición se necesitan a lo sumo tres posiciones de la matriz $(k-1)$ -ésima., La columna k no cambia., La fila k no cambia.

Pregunta 20

Incorrecta

Se puntúa 0,00 sobre 1,00

El coste de obtener el máximo de un AVL con N nodos está en

Seleccione una:

- ☐ a. $O(N \log N)$
- ☐ b. $O(\log N)$
- ☒ c. $O(1)$ ✖
- ☐ d. $O(N)$

Para buscar el máximo hay que bajar por el árbol siempre hacia la derecha hasta que no se pueda más. En el caso peor habría que bajar la altura del árbol, y como el árbol es equilibrado, esta es proporcional al logaritmo del número de nodos.

La respuesta correcta es: $O(\log N)$

Pregunta 21

Sin contestar

Se puntúa como 0 sobre 1,00

Cuando en una cola de prioridad de mínimos con prioridades variables, implementada mediante un montículo, se cambia la prioridad de uno de los elementos, este siempre cambiará de posición en el montículo.

Seleccione una:

- ☐ a. Verdadero
- ☐ b. Falso

Falso. No necesariamente tiene que cambiar de posición. Solo cambiará si la prioridad se hace menor que su padre o mayor que el menor de los hijos.

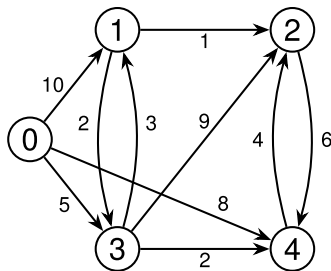
La respuesta correcta es: Falso

Pregunta 22

Correcta

Se puntúa 1,00 sobre 1,00

Supón que ejecutamos el algoritmo de Dijkstra sobre este grafo utilizando como origen el vértice 0.



¿Cuál es el vértice anterior a 2 en su camino mínimo?

Seleccione una:

- ☐ a. Ninguno, porque no es alcanzable desde el origen.
- ☐ b. 4
- ☐ c. 3
- ☒ d. 1 ✓

El camino mínimo del vértice 0 al vértice 2 es $0 \rightarrow 3 \rightarrow 1 \rightarrow 2$. El penúltimo vértice es el 1.

La respuesta correcta es: 1

Pregunta 23

Correcta

Se puntúa 1,00 sobre 1,00

Utilizando las estructuras de partición podemos detectar si un grafo es acíclico. Si están implementadas con *unión rápida por tamaño y compresión de caminos*, ¿cuál sería la complejidad de dicho algoritmo si el grafo tiene V vértices y A aristas?

Seleccione una:

- ☐ a. $O(V \log V)$
- ☒ b. $O(V + A \lg^* V)$ ✓
- ☐ c. $O(V + A \log V)$
- ☐ d. $O(VA)$

El algoritmo consiste en recorrer todas las aristas haciendo unión de sus extremos. Si los extremos de una arista ya pertenecen a la misma clase de equivalencia el grafo contiene un ciclo. Se realizan A operaciones de unión sobre V elementos, por lo que utilizando unión rápida por tamaño y compresión de caminos el coste de todas las operaciones está en $O(V + A \lg^* V)$.

La respuesta correcta es: $O(V + A \lg^* V)$

Pregunta 24

Correcta

Se puntúa 1,00 sobre 1,00

En un *grafo dirigido*, para encontrar el camino más corto (con menos aristas) entre dos vértices se utiliza

Seleccione una:

- ☒ a. búsqueda en anchura. ✓
- ☐ b. tanto búsqueda en profundidad como en anchura.
- ☐ c. ni búsqueda en profundidad ni en anchura.
- ☐ d. búsqueda en profundidad.

La búsqueda en anchura, al ir recorriendo los vértices por distancias crecientes desde el origen, garantiza que encuentra el camino con menos aristas.

La respuesta correcta es: búsqueda en anchura.

Pregunta 25

Correcta

Se puntúa 1,00 sobre 1,00

Tenemos una pared en la que queremos colocar el mayor número de rollos con papel pintado sin que se solapen. Los tramos de papel pintado los colocaremos en vertical.

Considerando que la longitud de cada rollo de papel es la misma (y la necesaria para cubrir completamente de arriba a abajo la altura de la pared), pero su anchura no. ¿Cuál de las siguientes estrategias voraces es correcta para resolver el problema anterior?

Seleccione una:

- ☐ a. Ordenamos los rollos por orden creciente en su longitud y elegimos el primero libre que quepa
- ☐ b. Elegimos cada vez un rollo cualquiera de los que tenemos disponibles
- ☒ c. Ordenamos los rollos por orden creciente de anchura y cogemos el primero libre que quepa ✓
Cierto. Intuitivamente estamos cogiendo aquellos rollos que más hueco libre nos dejan para después. Se puede demostrar mediante el método de reducción de diferencias que dicha estrategia es óptima para resolver el problema.
- ☐ d. Ninguna de las anteriores.

a. Falso. Incorrecta. La longitud de los rollos es la misma por lo que esa ordenación no nos ayuda. Por ejemplo, en una pared de 70cm y tres rollos de 70cm, 50cm y 20cm. La estrategia propuesta cogería sólo el primero y la óptima los dos últimos.

b. Falso. Incorrecta. Si elegimos en cada ocasión un rollo aleatorio no está garantizado obtener el óptimo. Por ejemplo, en una pared de 70cm y tres rollos de 70cm, 50cm y 20cm. La estrategia propuesta podría coger únicamente el primero y la óptima los dos últimos.

c. Cierto. Intuitivamente estamos cogiendo aquellos rollos que más hueco libre nos dejan para después. Se puede demostrar mediante el método de reducción de diferencias que dicha estrategia es óptima para resolver el problema.

d. Falso. La respuesta correcta es: Ordenamos los rollos por orden creciente de anchura y cogemos el primero libre que quepa

La respuesta correcta es: Ordenamos los rollos por orden creciente de anchura y cogemos el primero libre que quepa